BAHRIA UNIVERSITY

Web Systems & Technologies

Project Submission

SUBMITTED TO

Sir Adnan Jelani

Submitted By

Hassnain Abbas 01-135231-050

Suleman Tariq 01-135231-086

Asad Ullah 01-135231-015

CLASS: BSIT 5A

Table of Contents

1

D&Z Cartique: Comprehensive Technical Report 3

Project Overview 3

Team Members: 3

Technology Stack: 3

Core Features: 3

1. Functional Documentation 3

1.1 User Authentication 3

1.2 Product Catalog with Categories 4

1.3 Persistent Shopping Cart 5

1.4 Checkout with Order Tracking 5

1.5 Admin Dashboard 6

2. Firestore Database Schema 6

Collections and Fields: 6

2.1 Users 6

2.2 Products 7

2.3 Orders 7

3. Screenshot Sections 7

Homepage: Product Grid 7

User Authentication Pages 8

Cart and Checkout Flow 9

Order History Page 10

Admin Dashboard Interface (CRUD) 10

4. Glossary of Technical Terms 11

Conclusion 12

D&Z Cartique: Comprehensive Technical Report

Project Overview

Project Title: D&Z Cartique — E-commerce Web Platform

Website Link: https://dz-cartique.web.app/

Team Members:

- **Asad** — UI/UX Designer
- **Suleman** — Frontend Developer (Html + Redux)
- **Hassnain** — Backend Developer (Firebase: Auth & Firestore)

Technology Stack:

- **Frontend:** Html, CSS, JavaScript
- **Backend:** Firebase (Authentication & Firestore Database)
- **Live Website:** https://dz-cartique.web.app/

Core Features:

- User Authentication (Login / Register / Logout)
- Product Catalog with Categories
- Persistent Shopping Cart
- Checkout with Order Tracking
- Admin Dashboard (Manage Users / Products / Orders)

---

1. Functional Documentation

1.1 User Authentication

Description: Allows users to securely register, log in, and log out using Firebase Authentication.

Technical Implementation:

```
// firebaseAuth.js
```

```
import { getAuth, signInWithEmailAndPassword,
createUserWithEmailAndPassword, signOut } from "firebase/auth";

const auth = getAuth();

export const login = (email, password) =>
signInWithEmailAndPassword(auth, email, password);
export const register = (email, password) =>
createUserWithEmailAndPassword(auth, email, password);
export const logout = () => signOut(auth);
```

Data Flow Diagram:

```
[User] → [Html Form] → [Firebase Auth] ← [Firestore user data]
```

Error Handling:

- Invalid email/password: Display user-friendly error.
- Network failure: Retry prompt.

1.2 Product Catalog with Categories

Description: Displays a list of products with filtering by category.

Technical Implementation:

```
// productsSlice.js (Redux)
export const fetchProducts = createAsyncThunk('products/fetch', async
() => {
  const querySnapshot = await getDocs(collection(db, "products"));
  return querySnapshot.docs.map(doc => ({ id: doc.id, ...doc.data()
}));
});
```

Data Flow Diagram:

```
[Html Component] → [Redux Action] → [Firestore: products collection] →
[Redux Store] → [UI Render]
```

Error Handling:

- Show loading spinner while fetching.
- Show error message if fetch fails.

1.3 Persistent Shopping Cart

Description: Users can add items to cart and it persists across sessions using localStorage.

Technical Implementation:

```
// cartSlice.js
const initialState = JSON.parse(localStorage.getItem('cart')) || [];

export const cartSlice = createSlice({
  name: 'cart',
  initialState,
  reducers: {
    addToCart: (state, action) => {
      state.push(action.payload);
      localStorage.setItem('cart', JSON.stringify(state));
    },
    removeFromCart: (state, action) => {
      const updatedCart = state.filter(item => item.id !==
action.payload);
      localStorage.setItem('cart', JSON.stringify(updatedCart));
      return updatedCart;
    },
  },
});
```

Data Flow Diagram:

```
[User Action] → [Redux State] → [localStorage] ← [Session Restore]
```

Error Handling:

- Prevent adding out-of-stock products.
- Alert if localStorage is unavailable.

1.4 Checkout with Order Tracking

Description: Users can proceed to checkout, place orders, and track them.

Technical Implementation:

```
// placeOrder.js
export const placeOrder = async (orderData) => {
  const orderRef = await addDoc(collection(db, "orders"), {
    ...orderData,
    status: "Pending",
    createdAt: serverTimestamp(),
  });
```

```
  return orderRef.id;
};
```

Data Flow Diagram:

```
[Cart State] → [Checkout Form] → [Firestore: orders collection] →
[Order ID returned] → [UI Confirmation]
```

Error Handling:

- Disable button during payment processing.
- Show error message if order fails to save.

1.5 Admin Dashboard

Description: Admin users can manage products, orders, and users.

Technical Implementation:

```
// adminFunctions.js
export const deleteProduct = async (productId) => {
  await deleteDoc(doc(db, "products", productId));
};

export const updateOrderStatus = async (orderId, status) => {
  await updateDoc(doc(db, "orders", orderId), { status });
};
```

Data Flow Diagram:

```
[Admin Action] → [Form/Edit View] → [Firestore Update/Delete] →
[Updated UI State]
```

Error Handling:

- Confirm deletion prompts.
- Validate all form inputs.

---

2. Firestore Database Schema

Collections and Fields:

2.1 Users

```
{
  "uid": "user123",
```

```
  "email": "user@example.com",
  "role": "admin" | "customer",
  "createdAt": Timestamp
}
```

2.2 Products

```
{
  "name": "Shirt",
  "price": 1200,
  "category": "Clothing",
  "stock": 50,
  "imageUrl": "https://...",
  "description": "Cotton shirt..."
}
```

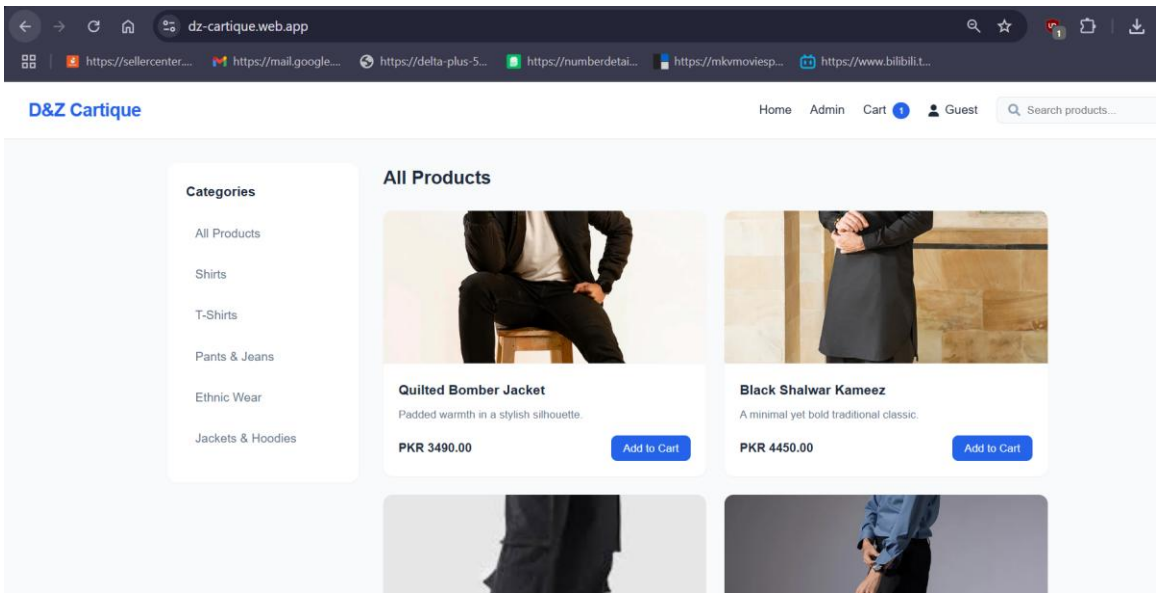
2.3 Orders

```
{
  "userId": "user123",
  "items": [{ "productId": "p1", "quantity": 2 }],
  "status": "Pending" | "Shipped" | "Delivered",
  "totalAmount": 2400,
  "createdAt": Timestamp
}
```

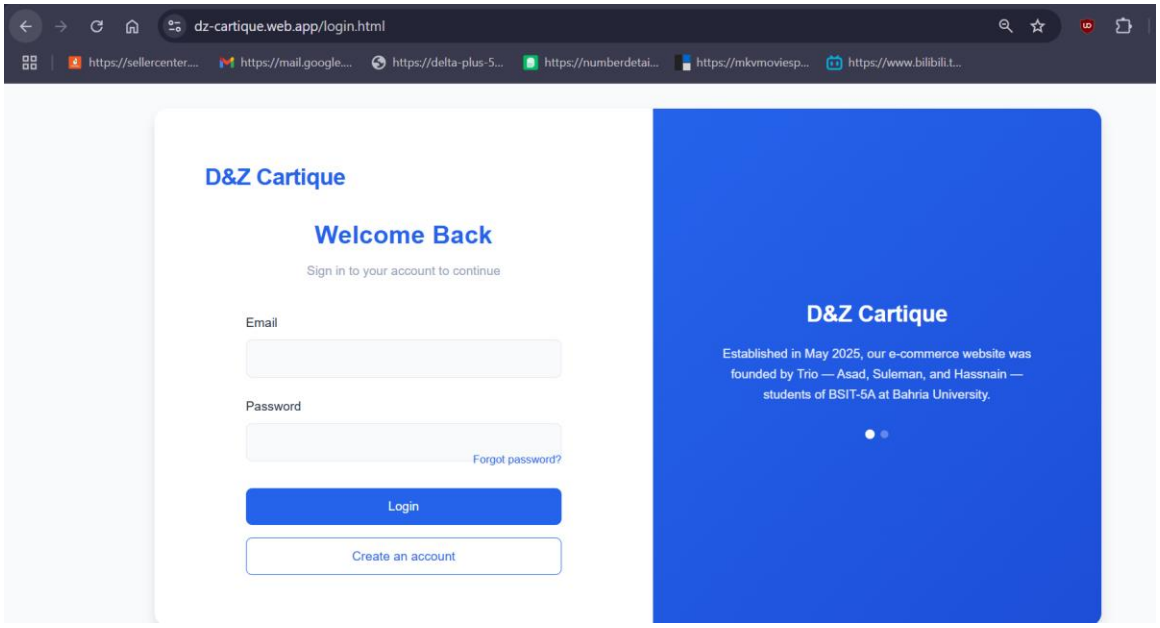
---

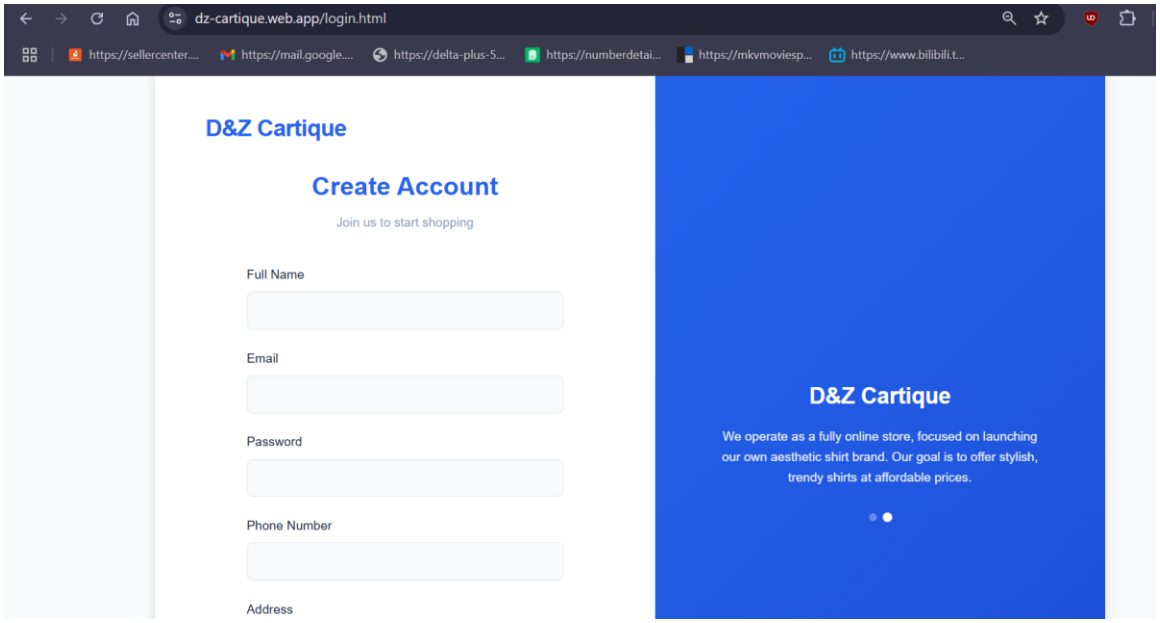
3. Screenshot Sections

Homepage: Product Grid

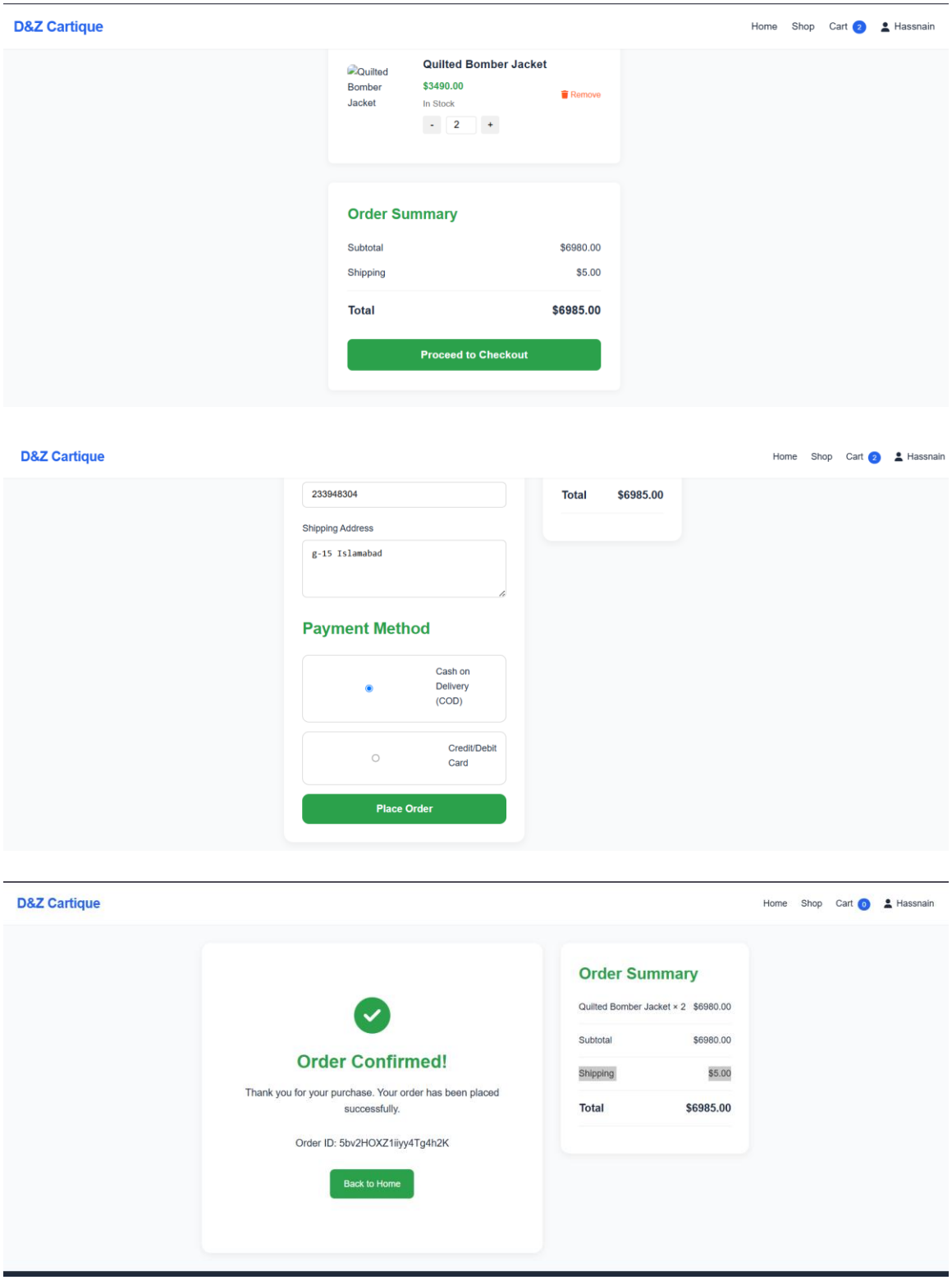


User Authentication Pages

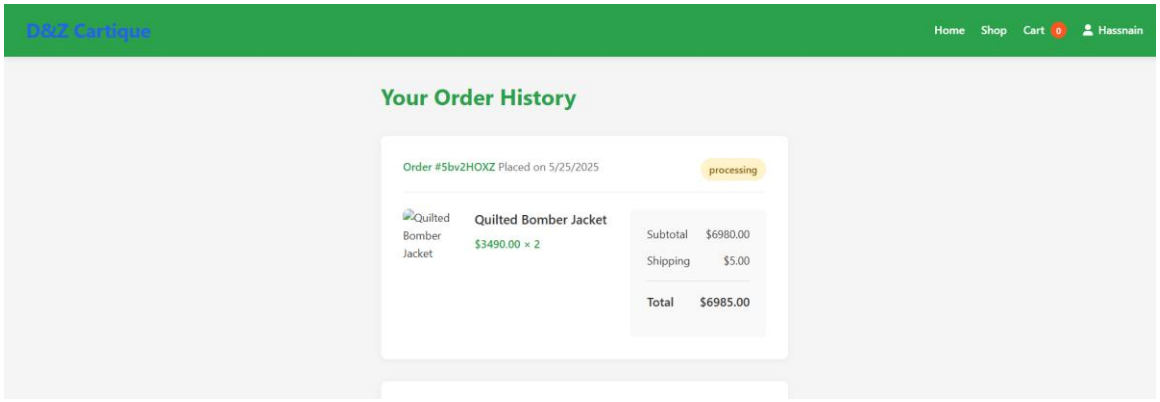




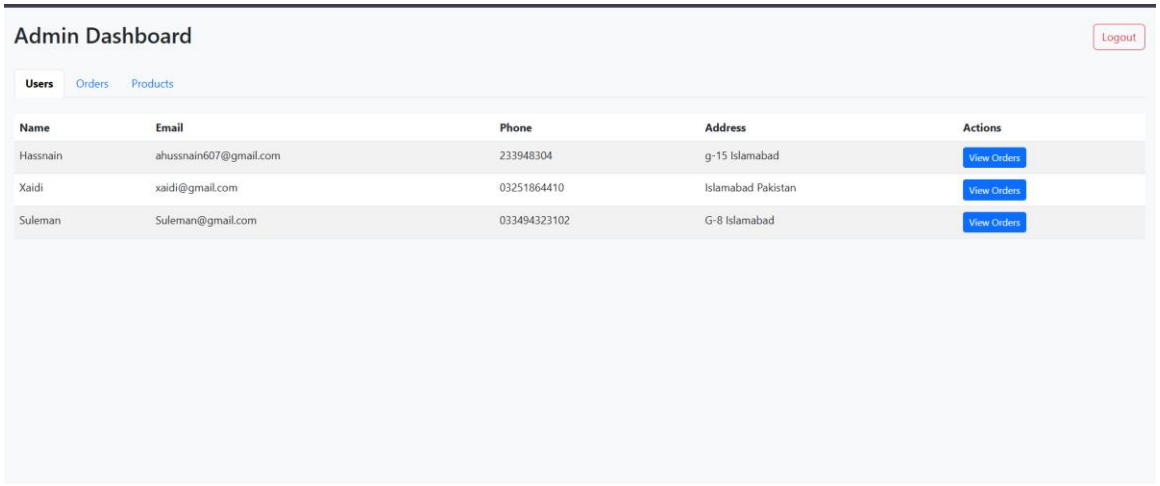
Cart and Checkout Flow

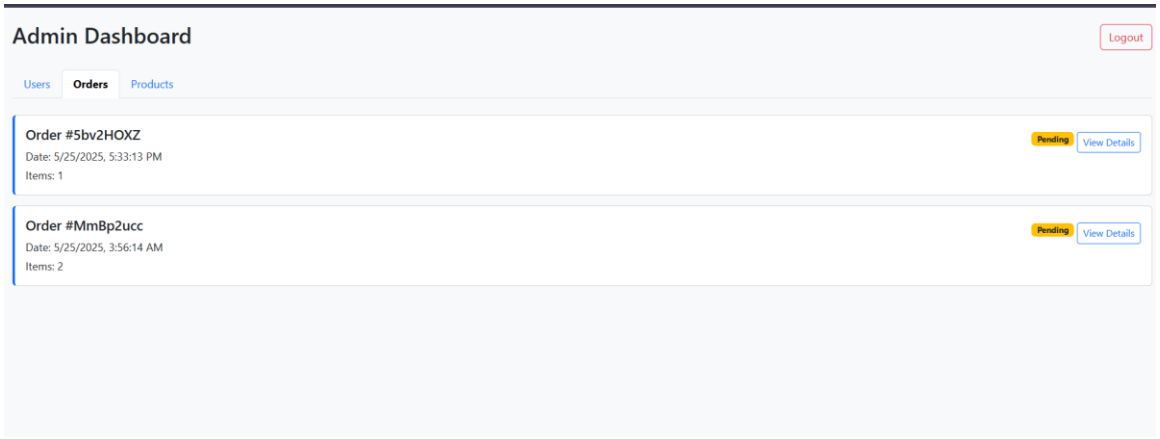


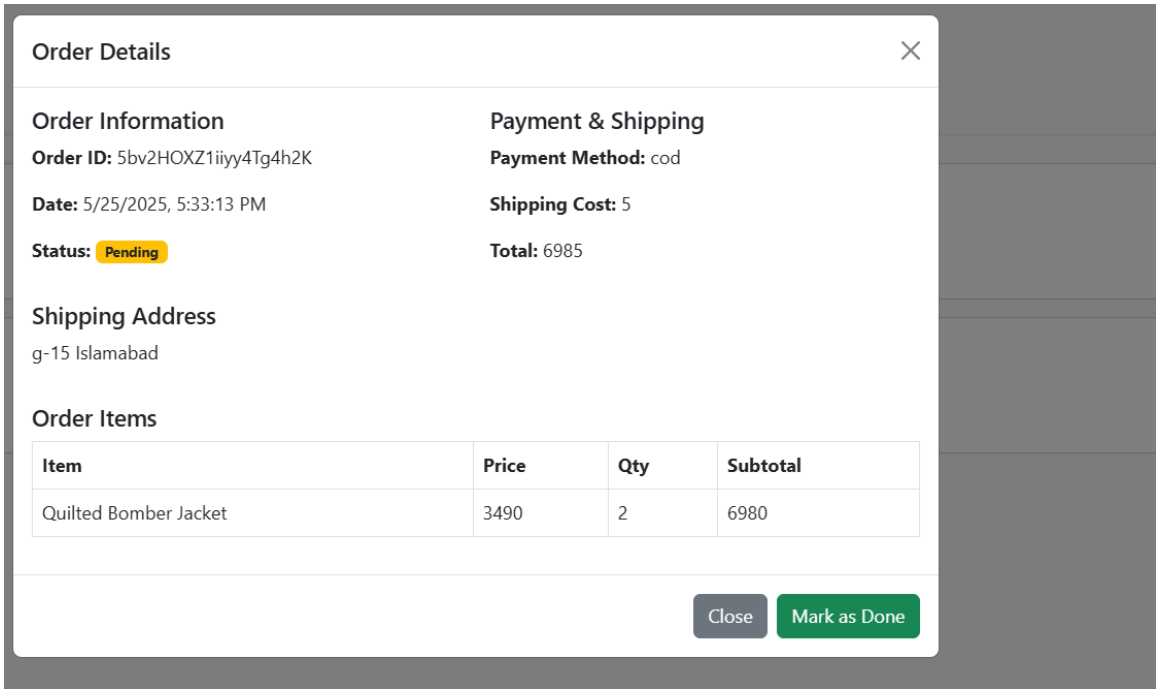
Order History Page

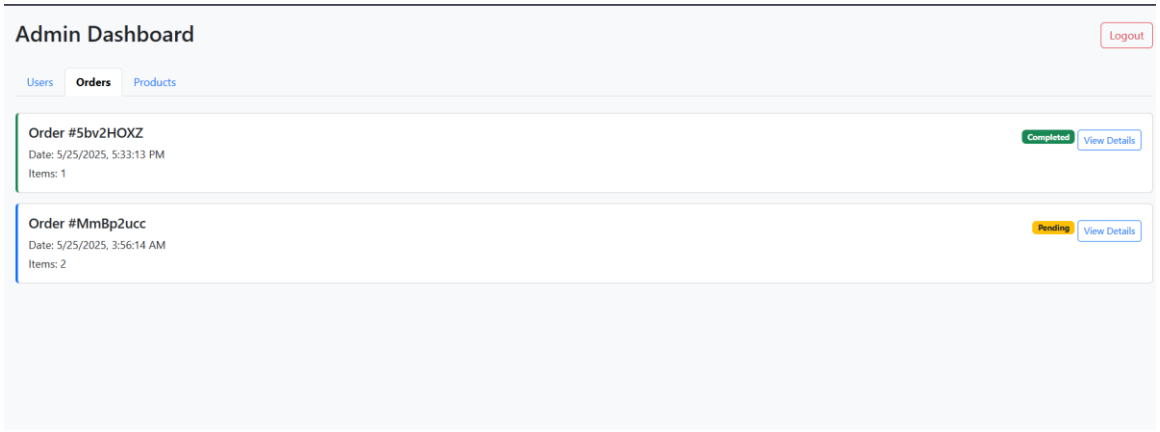


Admin Dashboard Interface (CRUD)









4. Glossary of Technical Terms

- **Firestore:** A NoSQL database by Firebase that stores data in collections and documents.
- **Redux:** A state management library used with Html to handle global app state.
- **AsyncThunk:** A middleware function in Redux Toolkit for handling async operations.
- **localStorage:** Browser storage used for saving data that persists even after a page reload.
- **CRUD:** Refers to Create, Read, Update, and Delete operations.
- **Authentication:** A process to verify the identity of users.

Conclusion

D&Z Cartique is a modern, scalable e-commerce web app built with Html and Firebase. It offers a seamless user experience from browsing products to order tracking and admin-level controls. The use of modern technologies ensures scalability, security, and ease of maintenance.

Note: Please insert screenshots in the labeled sections and format this report in Microsoft Word or Google Docs using appropriate headers and bullet styles.